

1. Relational Algebra (5 Core Operators)

Procedural

- User specifies **WHAT** data & **HOW** to retrieve it step-by-step.
- Selection ($\sigma_p(R)$):** Horizontal slice of rows satisfying condition p. Preserves degree, reduces cardinality.
 - Projection ($\pi_A(R)$):** Vertical slice of columns. **Automatically removes duplicate rows!** Degree = |A|.
 - Union ($R \cup S$):** Merges tuples. Requires **Union Compatibility** (same degree & matching attribute domains).
 - Set Difference ($R - S$):** Tuples in R but absent in S. Non-commutative ($R - S \neq S - R$). Union-compatible.
 - Cartesian Product ($R \times S$):** Pairs every row. $\text{deg} = m + n$, $\text{card} = m * n$.

2. Derived Joins & Division Operator

Joins & Math

- Theta Join ($R \bowtie_{\theta} S$):** $\sigma_{\theta}(R \times S)$. Equi-join uses equality (=).
- Natural Join ($R \bowtie S$):** Equi-join on common column names + projects out duplicate columns automatically.
- Outer Joins:** Preserves dangling unmatched tuples with NULLs.
 - Left Outer ($R \ltimes S$)
 - Right Outer ($R \rtimes S$)
 - Full Outer ($R \ltimes\!\!\!\bowtie S$)
- Division Operator ($R \div S$):** Solves "FOR ALL / EVERY" queries.

$$R(A, B) \div S(B) = \pi_A(R) - \pi_A(\pi_A(R) \times S - R)$$

3. Relational Calculus (TRC vs DRC) & Codd's Theorem

Declarative

Tuple Relational Calculus (TRC): $\{ t \mid P(t) \}$. Variable t represents an entire row. Uses quantifiers $\exists$ (Exists) and $\forall$ (For all). De Morgan: $\forall s(P(s)) \equiv \neg(\exists s(\neg P(s)))$.
Domain Relational Calculus (DRC): $\{ \langle x_1, x_2... \rangle \mid P \}$. Variables represent column attribute domains. Basis of visual QBE query languages.

Safe Expressions: A query is safe if it produces a finite set of tuples bounded within the active domain. Unsafe: $\{ t \mid \neg(t \in R) \}$.

CODD'S THEOREM: Relational Algebra = Safe TRC = Safe DRC

4. SQL Data Types & Storage

Storage

- CHAR(n):** Fixed-length string. Padded with trailing spaces.
- VARCHAR(n) / VARCHAR2:** Variable-length string. Space-efficient.
- INT, BIGINT:** Exact whole numbers without fractions.
- DECIMAL(p, s):** Fixed-point precision p with s decimal digits.
- DATE, TIMESTAMP:** YYYY-MM-DD calendar and microsecond timestamps.
- BLOB, CLOB:** Binary & Character Large Objects (Images, PDFs).

5. SQL Commands Classification

Taxonomy

- DDL (Data Definition):** Schema objects create/alter. Auto-committed! **CREATE**, **ALTER**, **DROP**, **TRUNCATE**, **RENAME**.
- DML (Data Manipulation):** Tuples manage. **SELECT**, **INSERT**, **UPDATE**, **DELETE**.
- DCL (Data Control):** Security. **GRANT**, **REVOKE**.
- TCL (Transaction Control):** Integrity. **COMMIT**, **ROLLBACK**, **SAVEPOINT**.

6. DROP vs TRUNCATE vs DELETE Comparison

10-Marker Must

Evaluation Parameter	DROP TABLE	TRUNCATE TABLE	DELETE FROM
Command Category	DDL (Data Definition)	DDL (Data Definition)	DML (Data Manipulation)
Schema Status	Schema + Data destroyed completely.	Data deleted; Schema preserved.	Data deleted; Schema preserved.
WHERE Clause	Not Applicable.	Not supported (all rows purged).	Supported (WHERE id = 5).
Rollback Capability	Cannot rollback (Auto-committed).	Cannot rollback (Auto-committed).	Can rollback (Logged in undo).
Execution Speed	Instant disk deallocation.	Lightning Fast (Resets HWM).	Slower (Row-by-row undo logging).
Triggers Fired	DROP trigger only.	No DELETE triggers fired!	Fires row-level DELETE triggers.

1. SQL DML Operations

DML Commands

- INSERT INTO:** Inserts single/bulk tuples.
`INSERT INTO Emp (ID, Name) VALUES (101, 'Aman');`
- UPDATE:** Modifies existing column values.
`UPDATE Emp SET Sal = Sal * 1.1 WHERE Dept = 'IT';`
Precaution: Missing WHERE updates all rows in table!
- DELETE FROM:** Purges candidate rows.
`DELETE FROM Emp WHERE Resigned = 1;`
- SELECT:** Declarative query data extraction.

2. SQL Integrity Constraints Hierarchy

Constraints

- NOT NULL:** Column cannot store NULL values.
- UNIQUE:** All rows distinct (allows NULLs in standard SQL).
- PRIMARY KEY:** NOT NULL + UNIQUE table identifier.
- FOREIGN KEY:** References parent table primary key.
- CHECK:** Custom condition validation (`CHECK (Salary > 0)`).
- DEFAULT:** Fallback value on empty insert.

3. Foreign Key Referential Actions (ON DELETE / ON UPDATE)

Integrity Rules

1. CASCADE: Deleting/updating parent row automatically deletes/updates corresponding child records.

2. SET NULL: Deleting parent row sets foreign key column in child to `NULL` (Child must allow NULLs).

3. SET DEFAULT: Resets child foreign key to default defined value.

4. RESTRICT / NO ACTION (Default): Rejects parent deletion with an error if dependent child records exist!

4. Physical SQL Clauses Execution Order (Engine Pipeline)

Engine Logic

1. FROM & JOIN → 2. WHERE → 3. GROUP BY → 4. HAVING → 5. SELECT → 6. DISTINCT → 7. ORDER BY → 8. LIMIT

Crucial Rule: WHERE clause executes BEFORE SELECT, so you **CANNOT use column aliases in WHERE!** ORDER BY executes AFTER SELECT, so aliases are perfectly valid in ORDER BY.

5. The 5 Aggregate Functions

Aggregates

- COUNT(*):** Counts total rows including NULL rows.
- COUNT(col):** Counts non-NULL column entries.
- SUM(col):** Sums non-NULL numeric values.
- AVG(col):** Arithmetic mean: Sum / non-NULL count.
- MIN / MAX(col):** Extremum values (numeric/string/date).
- Golden Rule:** All aggregates ignore NULLs (except COUNT(*)).

6. GROUP BY Projection Rule

Grouping

Partitions candidate rows into summary buckets.

Golden Projection Rule: When using GROUP BY, columns in the SELECT list MUST either:

- Be explicitly listed in the GROUP BY clause, OR
- Be wrapped inside an aggregate function (SUM, AVG, etc.).

7. WHERE vs HAVING (Top AKTU 10-Marker Comparison)

Exam Classic

Comparison Parameter	WHERE Clause	HAVING Clause
Operation Scope	Operates on individual rows (tuples) .	Operates on grouped summary buckets .
Execution Timing	Executes BEFORE GROUP BY partitioning.	Executes AFTER GROUP BY aggregation.
Aggregate Functions	Strictly Forbidden (<code>WHERE AVG(sal) > 50k</code> is ERROR).	Designed for aggregates (<code>HAVING AVG(sal) > 50k</code>).
Syntax Example	<code>SELECT Dept, AVG(Sal) FROM Emp WHERE Status='Active' GROUP BY Dept HAVING AVG(Sal) > 60000;</code>	Can appear without GROUP BY (filters whole table aggregate).

1. SQL Joins & Set Operations

Joins & Sets

- INNER JOIN:** Returns only matching records.
- SELF JOIN:** Joins table with itself (Employee & Manager).
- OUTER JOINS:** Retains unmatched dangling rows with NULL.
 - LEFT OUTER • RIGHT OUTER • FULL OUTER.
- UNION vs UNION ALL:**
 - *UNION*: Deduplicates results (internal sort overhead).
 - *UNION ALL*: Preserves duplicates. Extremely fast!
- INTERSECT:** Common rows • **MINUS:** Subtracts rows.

2. Nested vs Correlated Subqueries

Subqueries

- Scalar Subquery:** Returns single value (=, >, <).
- Multi-Row:** *IN*, *> ANY* (> MIN), *> ALL* (> MAX).
- Correlated Subquery:** Inner query references outer row attributes; executes row-by-row (O(N*M) nested loop).
- EXISTS vs NOT EXISTS:** Boolean short-circuit evaluation! Stops immediately upon finding first match. Faster than IN on large indexed tables.

3. SQL Views & Security

Virtual Views

- View:** Virtual table; stores query in data dictionary.
- Benefits:** Column security masking, complex join simplification, logical data independence.
- Updatable View Rules:** Single base table, contains Primary Key, NO aggregates, NO GROUP BY/DISTINCT.
- WITH CHECK OPTION:** Rejects inserts/updates violating the view's WHERE predicate filter.
- Materialized View:** Physically stored snapshot on disk (OLAP).

4. Database Indexes (B-Tree)

Storage Index

- Clustered Index:** Data physically sorted on disk in index order. Leaf nodes contain actual rows. **Only 1 per table** (Auto on PK).
- Non-Clustered:** Separate B-Tree. Leaf nodes contain Key + Row Pointer. Multiple allowed.
- Dense Index:** Index entry for EVERY row (Any table).
- Sparse Index:** 1 entry per disk block (Requires sorted table!).
- Trade-off:** Fast SELECT O(log N) vs slower INSERT/UPDATE.

5. PL/SQL Cursors (Memory Pointer)

PL/SQL

- Implicit Cursors:** Auto-managed by DBMS for all DML. Attributes: *SQL%FOUND*, *SQL%NOTFOUND*, *SQL%ROWCOUNT*, *SQL%ISOPEN*.
- Explicit Cursors (4-Step Lifecycle):**
 1. *DECLARE* (define query) → 2. *OPEN* (execute active set) → 3. *FETCH* (load into variables) → 4. *CLOSE* (free memory).
- Cursor FOR Loop:** Auto opens, fetches & closes cursor.

6. Database Triggers (ECA Model)

Automation

- Event:** INSERT, UPDATE, DELETE on table.
- Timing:** *BEFORE* (validation) vs *AFTER* (auditing/logs).
- Granularity:** *FOR EACH ROW* (Row-level) vs Statement-level.
- Pseudo-records:** *:NEW* (incoming values) and *:OLD* (pre-update values). Reference: *:NEW.Salary*.
- AKTU Example:** Prevent salary reduction:
`IF :NEW.Salary < :OLD.Salary THEN RAISE_APPLICATION_ERROR(-20001, 'Err'); END IF;`

7. Solved AKTU Examination Problem: Suppliers-Parts Database

10-Marker PYQ

Target Query Goal	Relational Algebra Solution	SQL Query Solution
Suppliers of RED parts	$\pi_{\text{name}}(\text{Suppliers} \bowtie \pi_{\text{sid}}(\text{Cat} \bowtie \sigma_{\text{color}='RED'}(\text{Parts})))$	<code>SELECT DISTINCT S.sname FROM Suppliers S JOIN Catalog C ON S.sid=C.sid JOIN Parts P ON C.pid=P.pid WHERE P.color='RED';</code>
Suppliers of ALL parts (Division)	$\pi_{\text{name}}(\text{Suppliers} \bowtie (\pi_{\text{sid}, \text{pid}}(\text{Cat}) \div \pi_{\text{pid}}(\text{Parts})))$	<code>SELECT S.sname FROM Suppliers S JOIN Catalog C ON S.sid=C.sid GROUP BY S.sid, S.sname HAVING COUNT(DISTINCT C.pid) = (SELECT COUNT(*) FROM Parts);</code>
Suppliers of NO Red parts	$\pi_{\text{sid}}(\text{Suppliers}) - \pi_{\text{sid}}(\text{Cat} \bowtie \sigma_{\text{color}='RED'}(\text{Parts}))$	<code>SELECT sname FROM Suppliers WHERE sid NOT IN (SELECT C.sid FROM Catalog C JOIN Parts P ON C.pid=P.pid WHERE P.color='RED');</code>

CS Fundamental Curricula • DBMS Unit 2

Page 3 of 3 (Joins, Views, Indexes & PYQs)

AKTU BCS501 Ready